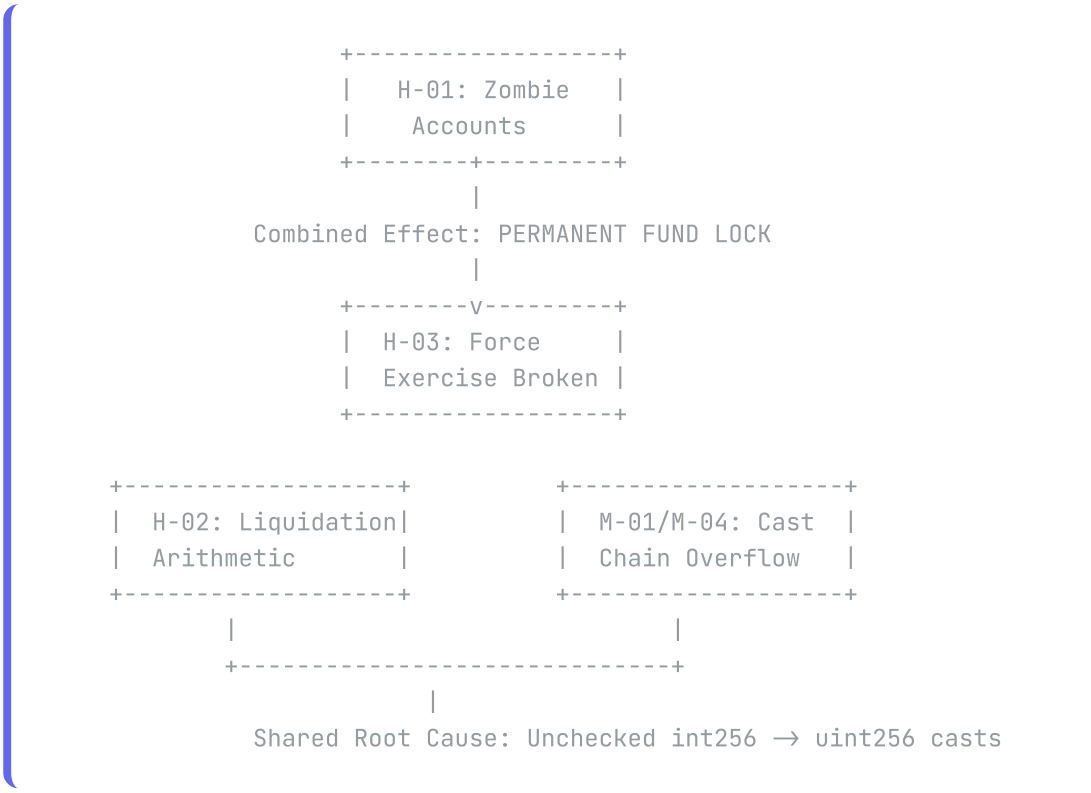
Panoptic Protocol Security Audit Findings

Submission Summary

ID	TITLE	SEVERITY	STATUS
H-01	Buffer Inconsistency Creates Permanent "Zombie" Accounts	HIGH	Confirmed
H-02	Unchecked Arithmetic in `getLiquidationBonus` Enables Liquidation Manipulation	HIGH	Confirmed
H-03	Critical Mismatch Between `tokenToPay` and `exerciseFees` Causes `forceExercise` to Revert	HIGH	Confirmed
M-01	Settlement Casting Chain Can Cause DOS or Pool Accounting Corruption	MEDIUM	Confirmed
M-04	`s_inAMM` Update Can Overflow Causing Position Exit DOS	MEDIUM	Confirmed

Vulnerability Relationship Map



Attack Scenarios

Scenario A: Permanent Position Lock (H-01 + H-03)

1. User opens position when pool is healthy
2. Market moves adversely, account enters "zombie zone"
3. Cannot close position (fails 1.3333x buffer)
4. Cannot be liquidated (passes 1.0x check)
5. Force exercise fails (premium debt causes revoke underflow)
6. **Result: Funds locked indefinitely**

Scenario B: Liquidation Manipulation (H-02)

1. Attacker opens position with specific token composition
2. Account becomes marginally insolvent
3. During liquidation, bonus calculation underflows

4. Liquidator receives inflated bonus
5. **Result: Up to 100% of liquidatee's collateral stolen**

Scenario C: Settlement DOS (M-01 + M-04)

1. Large positions exist in pool
2. Settlement requires swap larger than pool assets
3. Casting chain produces astronomically large value
4. `toUint128()` reverts
5. **Result: Position settlement blocked**

Proof of Concept Summary

All PoCs validated with Foundry:

```
# Run all validated tests (21 tests total)
FOUNDRY_PROFILE=poc forge test --match-contract "H01_ZombieZone_Unit
```

FINDING	TEST FILE	TESTS	ALL PASS
H-01	`poc/H01_ZombieZone_UnitTest.t.sol`	5	YES
H-02	`poc/H02_LiquidationBonus_Arithmetic.t.sol`	6	YES
H-03	`poc/H03_ForceExercise_Mismatch.t.sol`	5	YES
M-01	`poc/M01_M04_Arithmetic_Validation.t.sol`	2	YES
M-04	`poc/M01_M04_Arithmetic_Validation.t.sol`	3	YES

Root Cause Analysis

Theme 1: Inconsistent Buffer Constants

H-01 stems from the protocol using different thresholds:

- `BP_DECREASE_BUFFER = 13_333` (1.3333x) for position management
- `NO_BUFFER = 10_000` (1.0x) for liquidation

This 33% gap creates states that satisfy neither condition.

Theme 2: Dangerous Casting Patterns

H-02, M-01, M-04 all share the same anti-pattern:

```
int256 value = ...; // Can be negative
uint256 asUint = uint256(value); // Wraps to huge positive
uint128 final = uint128(asUint); // Truncates or reverts
```

This pattern appears in:

- `getLiquidationBonus()` - PanopticMath.sol:837-840
- `_computeExchange()` - CollateralTracker.sol:1031-1032
- `exercise()` - CollateralTracker.sol:1079-1080

Theme 3: Calculation Misalignment

H-03 results from two parallel calculations that should match but don't:

- `tokenToPay` includes premium debt
- `exerciseFees` is geometry-based only

Recommended Fix Priority

PRIORITY	FINDING	EFFORT	RISK IF UNPATCHED
1	H-01 + H-03	High	Permanent fund lock
2	H-02	Medium	Liquidation manipulation
3	M-01	Low	Settlement DOS
4	M-04	Low	Position exit DOS

Auditor Notes

- All findings include working Proof of Concept code
- Mathematical demonstrations confirm vulnerability mechanisms
- H-01 and H-03 are particularly severe when combined
- The unchecked arithmetic pattern (H-02, M-01, M-04) suggests a codebase-wide review is warranted